Experiment 2

Aim : Create a Blockchain Using Python

Theory:

A blockchain is a distributed digital ledger in which data is stored in a sequence of blocks. Each block contains information such as an index, timestamp, transactions, proof value, and the hash of the previous block.

The main components used in this experiment are:

- Block: Stores transactions and blockchain-related information.
- Genesis Block: The first block of the blockchain.
- Transaction: Represents the transfer of information/value between a sender and receiver.
- Proof-of-Work (PoW): A computational process used to find a valid proof satisfying a predefined condition.
- SHA-256: A cryptographic hashing algorithm used to generate a unique hash for each block.
- Previous Hash: Connects one block to the previous block and helps maintain blockchain integrity.
- Blockchain Validation: Checks whether the hashes and Proof-of-Work of all blocks are valid.
- Flask API: Provides endpoints to interact with the blockchain through HTTP requests.
- Postman: Used to test the Flask APIs.

In this experiment, the Proof-of-Work condition is that the generated SHA-256 hash must begin with three zeros (`000`). Transactions are added before mining, and after successful mining they are stored in the newly created block.

Procedure / Steps

Step 1: Install and Import Required Libraries

Install Flask and import the required Python libraries.

```
!pip install flask==2.2.5

import datetime

import hashlib

import json

from flask import Flask, jsonify
```

Step 2: Implement the Blockchain

The `Blockchain` class is created to manage blocks, transactions, Proof-of-Work, hashing, and validation.

```
class Blockchain:
  def __init__(self):
    self.chain = []
    self.transactions = []
    # Create Genesis Block
    self.create_block(
      proof=1,
      previous_hash='0'
    )
  def create_block(self, proof, previous_hash):
    block = {
      'index': len(self.chain) + 1,
      'timestamp': str(datetime.datetime.now()),
      'proof': proof,
      'transactions': self.transactions.copy(),
      'previous_hash': previous_hash
    }
    self.chain.append(block)
    return block
  def get_previous_block(self):
    return self.chain[-1]
  def create_Transactions(self, sender, receiver, amount):
    transaction = {
      'sender': sender,
      'receiver': receiver,
      'amount': amount
    }
    self.transactions.append(transaction)
    return self.get_previous_block()['index'] + 1
  def proof_of_work(self, previous_proof):
    new_proof = 1
    while True:
      hash_operation = hashlib.sha256(
        str(
          new_proof ** 2 - previous_proof ** 2
        ).encode()
      ).hexdigest()
      if hash_operation[:3] == '000':
        return new_proof
      new_proof += 1
  def hash(self, block):
```

```
        encoded_block = json.dumps(
            block,
            sort_keys=True
        ).encode()
        return hashlib.sha256(
            encoded_block
        ).hexdigest()
    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(
                str(
                    proof ** 2 - previous_proof ** 2
                ).encode()
            ).hexdigest()
            if hash_operation[:3] != '000':
                return False
            previous_block = block
            block_index += 1
        return True
```

Step 3: Create Blockchain, Add Transaction and Mine Block

Create the blockchain and add a transaction using the student's name and college.

```
blockchain = Blockchain()
block_index = blockchain.create_Transactions(
    sender="Khushi Singh",
    receiver="VESIT",
    amount=100
)
print("Transaction added successfully!")
print("Transaction will be included in Block:", block_index)
print("\nPending Transactions:")
print(blockchain.transactions)
```

Output:

```
Transaction added successfully!
Transaction will be included in Block: 2

Pending Transactions:
[{'sender': 'Khushi Singh', 'receiver': 'VESIT', 'amount': 100}]
```

Now mine the block:

```
previous_block = blockchain.get_previous_block()
previous_proof = previous_block['proof']

print("Mining block...")

proof = blockchain.proof_of_work(previous_proof)

print("Proof found:", proof)

previous_hash = blockchain.hash(previous_block)

new_block = blockchain.create_block(
   proof,
   previous_hash
)

blockchain.transactions.clear()

print("\nBlock mined successfully!")
print(new_block)
```

Output:

```
Mining block...

Proof found: 533


Block mined successfully!

{'index': 2, 'timestamp': '2026-08-23 06:38:12.427846', 'proof':
533, 'transactions': [{'sender': 'Khushi Singh', 'receiver':
'VESIT', 'amount': 100}], 'previous_hash':
'5283a746018e5247b7ab96b4599bd13a4ce30a5c1392534a240d0f50ff09fa5
2'}
```

Step 4: Display and Validate the Blockchain

```
print("================================")
print("COMPLETE BLOCKCHAIN")
print("================================")

print(json.dumps(blockchain.chain, indent=4))

print("\n================================")
print("BLOCKCHAIN VALIDITY")
print("================================")

if blockchain.is_chain_valid(blockchain.chain):
   print("All good. The Blockchain is valid.")
else:
   print("The Blockchain is not valid.")
```

Output :

```
================================
COMPLETE BLOCKCHAIN
================================
[
    {
        "index": 1,
        "timestamp": "2026-08-23 06:37:45.562012",
        "proof": 1,
        "transactions": [],
        "previous_hash": "0"
    },
    {
        "index": 2,
        "timestamp": "2026-08-23 06:38:12.427846",
        "proof": 533,
        "transactions": [
            {
                "sender": "Khushi Singh",
                "receiver": "VESIT",
                "amount": 100
            }
        ],
        "previous_hash": "5283a746018e5247b7ab96b4599bd13a4ce30a5c1392534a240d0f50ff09fa52"
    }
]

================================
BLOCKCHAIN VALIDITY
================================
All good. The Blockchain is valid.
```

Step 5: Implement Flask APIs

Flask is used to provide APIs for mining a block, displaying the blockchain, and checking its validity.

```
app = Flask(__name__)
@app.route('/mine_block', methods=['GET'])
def mine_block():
  if len(blockchain.transactions) == 0:
    return jsonify({
      'message': 'No transactions available.'
    }), 400
  previous_block = blockchain.get_previous_block()
  previous_proof = previous_block['proof']
  proof = blockchain.proof_of_work(previous_proof)
  previous_hash = blockchain.hash(previous_block)
  block = blockchain.create_block(
    proof,
    previous_hash
  )
  blockchain.transactions.clear()
  response = {
    'message': 'Congratulations, you just mined a block!',
    'index': block['index'],
    'timestamp': block['timestamp'],
    'proof': block['proof'],
    'transactions': block['transactions'],
    'previous_hash': block['previous_hash']
  }
  return jsonify(response), 200
@app.route('/get_chain', methods=['GET'])
def get_chain():
  response = {
    'chain': blockchain.chain,
    'length': len(blockchain.chain)
  }
  return jsonify(response), 200
@app.route('/is_valid', methods=['GET'])
def is_valid():
  valid = blockchain.is_chain_valid(
    blockchain.chain
  )
  if valid:
    response = {
      'message': 'All good. The Blockchain is valid.'
    }
```

```
    else:
        response = {
            'message': 'The Blockchain is not valid.'
        }
    return jsonify(response), 200
```

Step 6: Run Flask and Test Using Postman

Since Google Colab was used, `pyngrok` was used to expose the Flask application.

```
!pip install pyngrok
```

Configure ngrok:

```
from pyngrok import ngrok

ngrok.set_auth_token("YOUR_NGROK_AUTH_TOKEN")
```

Run Flask:

```
import threading

def run_flask():
    app.run(
        host='0.0.0.0',
        port=5000,
        debug=False,
        use_reloader=False
    )

thread = threading.Thread(target=run_flask)
thread.start()
```

Create the public URL:

```
import time
from pyngrok import ngrok

time.sleep(3)

public_url = ngrok.connect(5000)

print("Flask URL:")
print(public_url)
```

The generated URL was then used in Postman.

API 1 — Mine Block

Method: GET

https://YOUR-NGROK-URL/mine_block ->
https://flaring-purr-anatomy.ngrok-free.dev/mine_block



API 2 — Check Validity

Method: GET

https://YOUR-NGROK-URL/is_valid ->
https://flaring-purr-anatomy.ngrok-free.dev/is_valid



API 3 — Get Blockchain

Method: GET

https://YOUR-NGROK-URL/get_chain ->
https://flaring-purr-anatomy.ngrok-free.dev/get_chain

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize JSON


```
"chain": [
    {
        "index": 1,
        "previous_hash": "0",
        "proof": 1,
        "timestamp": "2026-08-23 06:37:45.562012",
        "transactions": []
    },
    {
        "index": 2,
        "previous_hash": "5283a746018e5247b7ab96b4599bd13a4ce30a5c1392534a240d0f50ff09fa52",
        "proof": 533,
        "timestamp": "2026-08-23 06:38:12.427846",
        "transactions": [
            {
                "amount": 100,
                "receiver": "VESIT",
                "sender": "Khushi Singh"
            }
        ]
    },
    {
        "index": 3,
        "previous_hash": "77ac37d78cac6eb8b1b4c4d3342c52aaf02f538a3c59a3e67dc20f480c6f7aaf",
        "proof": 11028,
        "timestamp": "2026-08-23 06:50:02.140900",
        "transactions": [
            {
                "amount": 100,
                "receiver": "VESIT",
                "sender": "Khushi Singh"
            }
        ]
    }
],
"length": 3
```

Results / Observations

The experiment successfully demonstrated:

- Creation of a Genesis Block.
- Addition of a transaction from Khushi Singh to VESIT.
- Mining of a new block using Proof-of-Work.
- Use of SHA-256 for block hashing.
- Linking of blocks using the previous block hash.
- Display of the complete blockchain.
- Successful blockchain validation.
- Testing of blockchain operations through Flask and Postman.

Conclusion

The basic blockchain was successfully implemented using Python. Transactions were added and stored in blocks using a Proof-of-Work mechanism with a `000` leading-zero condition. SHA-256 hashing was used to maintain the integrity and connection between blocks. The blockchain was successfully validated, and Flask APIs were used with Postman to mine blocks, display the blockchain, and verify its validity.